538 notebook — CELL 10 generative pool manual

Operator reference for `538_alex_tier1_model_and_fit.ipynb` **CELL 10** (Thread A: soft pre-sorting). This is **not** the **537** legacy score playground — see `sports/documents/537_Manual.md` for sort-and-chop, winner draws, and local-rank weights.

Terminology: Selection is domain-neutral (Army promotion, NBA draft, tenure). The generative stack uses `Y_selected`, `N_SELECTED`, etc. Empirical CELLs 0–6 may still use Army-specific outcome names.

Symbols: A_i = ability, T_j = team target mean. CELL 10 widget labels use Unicode subscripts (A_i, T_j) because ipywidgets does not render LaTeX in `description` text.

Related docs: `sports/documents/Tier1_Presorting_Design_Note.md` (design + calibration), `Alex_Tier1_Sequential_Model_Outline.md` §6A.

Quick start

1. Kernel cwd: **repo root** (directory containing `sports/`).
2. Run **CELL 0** (`RUN_CELL10 = True`).
3. Run **CELL 10** code (executes `sports/tier1_cell10_playground_run.py`).
4. Move sliders or click **Run / refresh plot**. Both **Plot A** (pool overlap) and **Plot B** (inverted-U preview) refresh together.
5. Widget values are **authoritative**; `sports/tier1_sim_config.py` is defaults only (**Load defaults from tier1_sim_config.py**).
6. Calibrate pools against **530** forensics (CELLs 5–8): overlap peak ≫ 1, median roster SD ~ 0.8. Tune selection sliders until Plot B shows a sensible inverted-U shape.

You do **not** need CELL 1–6 (real panel) for CELL 10.

What this cell does (one sentence)

Draw **team target means** T_j, draw **abilities** A_i, assign with **soft probabilities** $\pi_{ij} \propto f(A_i - T_j)$, then **select** top-$K$ players by score (LOO gap + ability by default). You get **two** live plots: **interval overlap** (530 CELL 8 analog) and **selection rate vs LOO Q bins** (inverted-U preview, same logic as CELL 12).

Core symbols (generative world)

Symbol	Meaning
J	Number of teams / pools (`N_TEAMS`, widget **Teams J**). Paper Directions 11: **J ≫** number of EDA ventile bins (often 8–20).
N	Total synthetic players = **J × roster size** (`N_INDIVIDUALS`).
A_i	Latent ability of player i **before** assignment (widget **A_i draw**).
T_j	**Fixed** target mean for team j (drawn once per refresh; not updated from realized roster mean).
τ	Assignment **temperature** (`ASSIGNMENT_TEMPERATURE`). Scales $(A_i - T_j)$ in the kernel. **Larger τ → more cross-team mixing and overlap.**
pool_id	Integer team label $0 \dots J-1$ after assignment.
team_target	T_j for the team player *i* was placed on.
ability (column)	Realized A_i on each row of the synthetic roster table.

Selection (Plot B): LOO pool quality `poolq_loo`, outcome `Y_selected`, bins on **player** LOO Q (not teams). **J** (teams) and **~12 bins** are different knobs — see §CELL 12 in this file.

Assignment algorithm (soft mode)

1. **Draw** $A_1, \dots, A_N$ from the chosen ability law.
2. **Draw** $T_1, \dots, T_J$ from the chosen target-mean law on $[T_j \text{ low}, T_j \text{ high}]$.
3. **Sequential placement** (random player order): for each player, compute weights
   $\tilde{\pi}_{ij} = f(A_i - T_j) \times (n_j + k)^{\alpha}$
   with f = Gaussian or Cauchy kernel; set weight 0 for teams already at **roster size**; sample one team; increment n_j.
4. **Benchmark overlay** (optional): re-use the **same** A_i vector, assign with **537 assortative sort-and-chop** (sort by ability, equal-count slices) — should give coverage ≈ 1 everywhere.

Implementation: `sports/tier1_pool_assignment.py` (`soft_assign`, `assign_sort_chop_benchmark`, `simulate_generative_rosters`).

Ability draw (A_i draw / `ABILITY_DRAW`)

Widget value	Config constant	Draw
normal_clipped	`ABILITY_DRAW = "normal_clipped"`	`Normal(ABILITY_MEAN, ABILITY_SD)` clipped to `[ABILITY_CLIP_LOW, ABILITY_CLIP_HIGH]`.
normal_plus_student_t	`ABILITY_DRAW = "normal_plus_student_t"`	Same normal draw + `ABILITY_STUDENT_T_SCALE * t(df=ABILITY_STUDENT_T_DF)`, then clip. Heavy tails → occasional "wrong-level" talent before assignment.
uniform_01	`ABILITY_DRAW = "uniform_01"`	`Uniform(0, 1)`.

Defaults for mean, SD, clip, and Student-*t* knobs: `tier1_sim_config.py` (not all exposed as sliders yet).

Target team means (Target T_j law / `TARGET_MEAN_DIST`)

Widget value	Config	Draw
uniform	`TARGET_MEAN_DIST = "uniform"`	$T_j \sim \text{Uniform}(T_j \text{ low}, T_j \text{ high})$.
normal_clipped	`TARGET_MEAN_DIST = "normal_clipped"`	$T_j \sim \mathcal{N}(\texttt{TARGET\_MEAN\_MU}, \texttt{TARGET\_MEAN\_SIGMA}^2)$ clipped to the same bounds.

T_j low / T_j high (widgets **T_j low**, **T_j high**) → `TARGET_MEAN_LOW`, `TARGET_MEAN_HIGH`.
Do **not** copy empirical college team means into T_j for the cross-domain minimal story (Alex) — use these ranges only to set **scale**, calibrated via overlap/SD plots.

Soft-match kernel (`Kernel` / `ASSIGNMENT_KERNEL`)

Value	Formula (up to proportionality)
gaussian	$\exp\left(-(A_i - T_j)^2/(2\tau^2)\right)$
cauchy	$1/\left(1 + ((A_i - T_j)/\tau)^2\right)$ — fatter tails → more distant teams still get weight.

Preferential attachment (`Pref. attach α` / `PREFERENTIAL_ALPHA`)

When **α > 0**, weights multiply by $(n_j + k)^\alpha$ where n_j is the current roster count for team j during sequential placement, and k = `PREFERENTIAL_K` (default 1.0 in config, not a slider).

- **α = 0** (default): pure soft match to fixed T_j.
- **α > 0**: rich-get-richer — teams that already have players attract more. Anchor stays on **fixed** T_j, not drifting pool mean.

CELL 10: widgets (top → bottom)

UI label (`description`)	Control	Maps to (`tier1_sim_config.py`)	Persist key (`tier1_cell10_playground_state.json`)
Teams J	IntSlider [4, **2500**] (`N_TEAMS_SLIDER_MAX`)	`N_TEAMS`	`n_teams`
Roster size	IntSlider [5, 40]	`ROSTER_SIZE`	`roster_size`
τ (temperature)	FloatSlider [0.05, 2.0]	`ASSIGNMENT_TEMPERATURE`	`tau`
Kernel	Dropdown	`ASSIGNMENT_KERNEL` (`gaussian` / `cauchy`)	`kernel`
Target T_j law	Dropdown	`TARGET_MEAN_DIST`	`target_dist`
T_j low	FloatSlider	`TARGET_MEAN_LOW`	`t_low`
T_j high	FloatSlider	`TARGET_MEAN_HIGH`	`t_high`
Pref. attach α	FloatSlider [0, 2]	`PREFERENTIAL_ALPHA`	`pref_alpha`
A_i draw	Dropdown	`ABILITY_DRAW`	`ability_draw`
Seed	IntSlider	`RANDOM_SEED` (config default; widget overrides per run)	`seed`
Overlay sort-and-chop (537 B)	Checkbox	— (benchmark only; Plot A)	`show_chop`
Show Plot A (overlap)	Checkbox	`SHOW_PLOT_A`	`show_plot_a`
LOO Q bins (#)	IntSlider [5, 30]	`GENERATIVE_N_BINS`	`n_bins`
Selections K	IntSlider [5, 2000]	`N_SELECTED`	`n_selected`
Selection score	Dropdown (labeled modes)	`SELECTION_SCORE_MODE`	`score_mode`
LOO-gap weight w	FloatSlider [0, 1]	`LOO_GAP_WEIGHT`	`loo_gap_weight`
(formula reminder)	HTML under w	—	updates live: score = w·(A–LOO Q)+(1–w)·A; **w=1** → gap only, **w=0** → ability only

UI label (`description`)	Control	Maps to (`tier1_sim_config.py`)	Persist key (`tier1_cell10_playground_state.json`)
Winner draw	Dropdown (labeled A/B/C)	`WINNER_SELECTION`	`winner_selection`
Load defaults from tier1_sim_config.py	Button	Reloads config file → copies into sliders → redraw	—
Run / refresh plot	Button	Redraw both plots	—

Persistence: After each successful plot, knob values are written to `sports/tier1_cell10_playground_state.json`. Delete that file to fall back to `tier1_sim_config.py` defaults on the next run (or use **Load defaults**).

Gate: `RUN_CELL10` in **CELL 0** — if `False`, CELL 10 code prints skip and does not launch widgets.

Plot A: interval overlap (530 CELL 8 analog)

Title: `538 CELL 10 – interval overlap (530 CELL 8 analog)`

Curve	Meaning
Blue (soft assign)	For each point on a fixed ability grid, count how many teams' **[min ability, max ability]** on that roster cover the point.
Red dashed (sort-and-chop)	Same count using **537 B** assignment on the **same** A_i draw.
Gray dotted horizontal at 1	"Disjoint partition" benchmark — only one team covers each point.

Summary line (HTML under controls):

Field	Meaning	Calibration hint (real data, 530)
coverage_peak	Maximum coverage over the grid	Soft: **≫ 1** (e.g. hundreds–thousands). Sort-chop: **≈ 1**.
median_pool_sd	Median within-roster SD of **ability**	Soft: aim **~ 0.8** (z-scale forensics). Sort-chop: often **much smaller** (tight slices).

Grid default: 81 points on $[-2, 2]$ (synthetic ability axis, not panel PPM unless you align scales deliberately).

Plot B: inverted-U preview (CELL 12 logic, live)

Title: `538 CELL 10 — inverted-U preview (N bins, K=...)`

Element	Meaning
X	Bin mean LOO `poolq_loo` (quantile bins by default)
Y	Mean `Y_selected` in bin
Summary line	`overall_rate` = K/N; `peak_bin_rate` = max bin rate

Shared code: `sports/tier1_generative_eda.py`. **CELL 12** re-runs the same pipeline as a static figure using saved CELL 10 state.

Tuning guide (what to turn first)

Goal	Turn
More overlap (higher blue peak)	**↑ τ**, **↑ J** (more teams with similar T_j), or **cauchy** kernel
Less overlap (closer to sort-chop)	**↓ τ**, **↓ J**, or **gaussian** with small τ
Wider spread of team "types"	Widen **T_j low / T_j high** or use **normal_clipped** with larger `TARGET_MEAN_SIGMA` (config)
Fatter talent tails before assignment	**normal_plus_student_t** or widen ability clip range in config
Rich-get-richer concentration	**↑ Pref. attach α** (small steps; easy to overpower soft match)
Stronger inverted-U (middle bins)	**↑ K**, **↑ LOO-gap weight w**, pools with enough LOO spread (τ, J)
Flatter selection curve	**↓ K**, score **ability** only, or winner **B** (stochastic)

Always compare Plot A to the **red dashed** overlay and **530 CELL 8** — the target is "blue like real rosters," not "blue like red." Use Plot B for selection shape while you adjust pools and K together.

File map

File	Role
`sports/538_alex_tier1_model_and_fit.ipynb`	CELL 10 UI entry (`exec` of playground script)
`sports/tier1_cell10_playground_run.py`	ipywidgets + dual plots
`sports/tier1_generative_eda.py`	Shared inverted-U bin table + figure
`sports/tier1_pool_assignment.py`	Assignment engine
`sports/tier1_sim_config.py`	Default constants
`sports/tier1_cell10_playground_state.json`	Last-used widget values (auto-written)
`sports/documents/Tier1_Presorting_Design_Note.md`	Design + 530 calibration checklist
`sports/530_sports_pipeline.ipynb`	Real-data forensics (CELLs 5–9)
`sports/documents/537_Manual.md`	Legacy simulation knobs (**different** CELL 10)

538 notebook parts (context)

Block	CELLs	Status
Empirical (realized pools)	0–6, 4B/4C	Wang ladder on `(team_id, season)` panel
Empirical inference	7 *(parked)*	Cluster SEs, FE — not active
Generative lab	**10**	Pools + selection widgets; Plot A + Plot B (this manual)
Generative replay	**11**	530 CELL 6 analog: mean vs pool SD + span histogram (uses CELL 10 state JSON)
Generative inverted-U	**12**	Same inverted-U as Plot B, static run (reads CELL 10 state JSON)

CELL 12 — selection knobs

Defaults live in `tier1_sim_config.py`; **CELL 10 widgets** override them and write `tier1_cell10_playground_state.json`. **CELL 12** reads that JSON (falls back to config if missing). Legacy persist key `n_promoted` is still read if present.

Constant / persist key	Default	Role
`GENERATIVE_N_BINS` / `n_bins`	12	**Plot bins** on LOO `poolq_loo` (not teams)
`GENERATIVE_POOLQ_BINNING` / `bin_mode`	`quantile`	Binning law (config only; not a CELL 10 slider yet)
`N_SELECTED` / `n_selected`	200	Selection slots K
`SELECTION_SCORE_MODE` / `score_mode`	`loo_gap_plus_ability`	Score for ranking
`LOO_GAP_WEIGHT` / `loo_gap_weight`	0.5	Gap weight when using gap mode
`WINNER_SELECTION` / `winner_selection`	`C`	Top-K (`C`), weighted (`A`), Bernoulli (`B`)

Last aligned with CELL 10–12 scripts and `tier1_sim_config.py` — 2026-05-19.